



UNIVERSITÀ DEGLI STUDI DI MILANO

FACOLTÀ DI SCIENZE E TECNOLOGIE

Reti di Calcolatori

RIP in C

Studente: Luca Loner

Matricola: 52240A

A.A.: 2025/2026

RIP in C

Luca Loner

Indice

1	Requisiti e obiettivi del progetto	1
1.1	Panoramica del protocollo RIP	1
1.1.1	Generalità	1
1.1.2	Limitazioni	1
1.2	Requisiti funzionali	2
1.2.1	Inizializzazione e configurazione	2
1.2.2	Gestione dei messaggi e scambio tabelle	2
1.2.3	Gestione del timeout e topologia dinamica	2
1.3	Assunzioni operative	3
1.3.1	Supporto limitato al protocollo	3
1.3.2	Configurazione esplicita delle reti	3
1.3.3	Semplificazione della logica di routing	3
1.3.4	Gestione dei tempi molto semplificata	3
1.3.5	Integrazione diretta con il kernel	3
1.3.6	Assunzione di ambiente controllato e sicuro	3
2	Architettura del sistema e modellazione dati	4
2.1	Modello I/O	4
2.2	Strutture dati principali	4
2.2.1	Messaggio RIP	4
2.2.2	Associazione CIDR - interfaccia	6
2.2.3	Entry del RIP database	7
2.2.4	RIP database	8
3	Analisi del codice	9
3.1	Avvio e ciclo principale	9
3.2	Caricamento della configurazione da file	12
3.2.1	load_config()	12
3.3	Gestione delle rotte	13
3.3.1	process_rip_packet()	13

3.3.2	process_route()	15
3.3.3	add_route()	17
3.3.4	find_route()	17
3.3.5	send_routing_table()	18
3.4	Timer e gestione del database	19
3.4.1	expire_timed_out_routes()	19
3.4.2	refresh_local_interface_routes()	20
3.4.3	graceful_shutdown()	23
3.5	Integrazione con il kernel	24
3.5.1	netlink_send_route_request()	24
3.5.2	netlink_update_route()	26
4	Test di funzionamento	28
4.1	Topologia della rete di test	28
4.2	Test di avvio e convergenza iniziale	28
4.2.1	Avvio dei nodi	28
4.2.2	Convergenza	29
4.3	Gestione dei guasti	31
4.3.1	Caduta del link RA-RC	31
4.3.2	Ripristino del link RA-RC	33
4.3.3	Caduta della rete 10.12.20.0/24	33
4.3.4	Cattura del traffico	35
5	Organizzazione e struttura del codice sorgente	36
6	Installazione e avvio	37
6.1	Installazione manuale (Makefile)	37
6.1.1	Requisiti di sistema	37
6.1.2	Clonazione della repository e compilazione	37
6.1.3	Configurazione del nodo	37
6.1.4	Esecuzione e pulizia	37
6.2	Docker	38
6.2.1	Immagine Docker	38
6.2.2	Configurazione dell'ambiente con Docker Compose	38
6.2.3	Avvio ed esecuzione	39
7	Sviluppi futuri	40

1 Requisiti e obiettivi del progetto

1.1 Panoramica del protocollo RIP

1.1.1 Generalità

RIP (definito RFC 2453) è un protocollo di tipo Interior Gateway Protocol (IGP) basato sull'algoritmo Distance-Vector di Bellman-Ford, concepito per operare all'interno di Autonomous System di estensione moderata, risultando inadatto a topologie su larga scala. RIP utilizza uno degli algoritmi di routing di una classe nota come algoritmi a distance vector, la cui origine è legata agli standard di rete Xerox. Tali protocolli erano sostanzialmente identici al Routing Information Protocol, con gli indirizzi Xerox sostituiti da un formato di indirizzo più generale, in grado di gestire IPv4 e altri tipi di indirizzo, e con aggiornamenti di routing limitati a uno ogni 30 secondi. A causa di questa somiglianza, il termine Routing Information Protocol (o semplicemente RIP) viene utilizzato per riferirsi sia al protocollo XNS che al protocollo utilizzato da routed.

1.1.2 Limitazioni

Come accennato in precedenza, RIP è destinato principalmente all'uso come protocollo IGP in reti di dimensioni moderate. Inoltre, è opportuno evidenziare le seguenti limitazioni specifiche:

- Il protocollo è limitato a reti il cui diametro è di massimo 15 hop in quanto i progettisti hanno ritenuto che la struttura di base del protocollo non fosse appropriata per reti più grandi. Tale limitazione presuppone che venga utilizzato un costo di 1 per ciascuna rete, cioè il modo in cui RIP viene normalmente configurato. Se l'amministratore di sistema sceglie di utilizzare pesi più elevati, il limite superiore di 15 può diventare un grosso problema nella logica dell'algoritmo decisionale.
- Il protocollo si basa sul "counting to infinity" per risolvere le situazioni di loop. Se la topologia in cui opera ha diverse centinaia di reti e si forma un loop di routing che le coinvolge tutte, la risoluzione del loop con tale meccanismo richiederebbe molto tempo (se la frequenza dei routing update fosse limitata) e larghezza di banda (se gli update venissero inviati ogni volta che vengono rilevate modifiche). Un loop di questo tipo consumerebbe una grande quantità risorse prima di essere corretto.
- Il protocollo utilizza "metriche" fisse per confrontare percorsi alternativi e non è quindi adatto a situazioni in cui è necessario scegliere i percorsi in base a parametri in tempo reale come delay, reliability o load. Le possibili estensioni per consentire la gestione di metriche di questo tipo potrebbero introdurre instabilità che il protocollo non è progettato per gestire fin dall'origine.

1.2 Requisiti funzionali

I requisiti funzionali sono suddivisi in quattro macro-scenari principali:

1.2.1 Inizializzazione e configurazione

Acquisizione dati dal file di configurazione

All'avvio, l'applicazione deve poter leggere la configurazione definita nel relativo file per acquisire le informazioni relative ai prefissi di rete corrispondenti alle interfacce su cui si desidera attivare il protocollo.

Inizializzazione del RIP Database

Una volta letto il file di configurazione l'applicazione deve essere in grado di gestire una tabella interna (da qui in poi denominata RIP Database) su cui vengono salvate tutte le informazioni e prese le decisioni necessarie alla logica dell'algoritmo. Tali informazioni devono poi essere riflesse sulla tabella di routing del kernel dell'host.

1.2.2 Gestione dei messaggi e scambio tabelle

Invio periodico dei messaggi di Update

Allo scadere del relativo timer, l'applicazione deve inviare periodicamente la propria tabella di routing a tutti i nodi adiacenti.

Elaborazione dei messaggi in ingresso

L'applicazione deve poter ricevere i messaggi inviati da altri nodi, processarli ed estrarne le informazioni per aggiornare il proprio database prendendo le corrette decisioni. Inoltre, l'applicazione deve essere in grado di poter rispondere ai messaggi di Request inviando immediatamente la propria tabella di routing aggiornata.

1.2.3 Gestione del timeout e topologia dinamica

Route expiration

L'applicazione deve gestire un timer per ciascuna rotta presente nel database e, qualora non riceva aggiornamenti da un vicino entro un tempo massimo prestabilito, deve impostare la metrica di quella rotta a 16 (16 = irraggiungibile).

Garbage collection

Dopo che una rotta viene contrassegnata come irraggiungibile (impostando la metrica a 16), viene fatto partire un timer allo scadere del quale tale rotta viene definitivamente eliminata dalla tabella di routing.

1.3 Assunzioni operative

1.3.1 Supporto limitato al protocollo

L'applicazione è pensata solo per IPv4/RIPv2, con multicast su 224.0.0.9 e porta UDP 520. Non implementa funzionalità extra dell'RFC come autenticazione, route tag, gestione avanzata di campi opzionali o compatibilità con varianti più ricche.

1.3.2 Configurazione esplicita delle reti

L'applicazione non implementa una rilevazione dinamica delle interfacce e delle rotte, all'avvio la lista di reti su cui attivare il protocollo viene letta da un file di configurazione statico.

1.3.3 Semplificazione della logica di routing

La logica implementata aggiorna le rotte in modo abbastanza diretto: una sola rotta per destinazione. Non c'è una gestione completa dei casi di parità, multipath o di tutte le regole di convergenza previste dall'RFC.

1.3.4 Gestione dei tempi molto semplificata

L'applicazione usa una gestione dei timer semplice tramite valori fissi per aggiornamenti periodici e scadenze, con jitter casuale. Non vengono implementati in modo completo gli aspetti di holddown, flush timer, timing più raffinati e comportamenti di transizione tipici di un router RFC-compliant.

1.3.5 Integrazione diretta con il kernel

L'applicazione installa direttamente le rotte nel kernel tramite socket netlink. Questo semplifica molto l'implementazione, ma non riflette il modello completo di un router che deve gestire anche ACL, preferenze e interazioni più complesse con la tabella di routing.

1.3.6 Assunzione di ambiente controllato e sicuro

Non ci sono meccanismi di autenticazione, filtraggio, sicurezza, o gestione di pacchetti anomali. Il progetto è stato ideato con l'obiettivo di simulare il comportamento base di RIP, ma non per un uso reale o interoperabile in produzione.

2 Architettura del sistema e modellazione dati

2.1 Modello I/O

L'applicazione utilizza un modello I/O event-driven e monothread, basato su un ciclo principale che utilizza la funzione `select()` per il multiplexing delle sorgenti di eventi. In particolare, il processo attende simultaneamente:

- Pacchetti RIP ricevuti sul socket UDP multicast;
- Eventi netlink provenienti dal kernel, relativi a cambiamenti di stato delle interfacce;
- La scadenza dei timer interni per l'invio degli aggiornamenti periodici.

Gli input di rete vengono gestiti tramite chiamate a `recvfrom()` per i messaggi RIP e `recvmsg()` per gli eventi netlink. Gli output vengono invece prodotti con `sendto()` per i pacchetti RIP e con operazioni netlink verso il kernel per l'installazione, l'aggiornamento o la rimozione delle rotte.

Il sistema non utilizza multithreading e tutta la logica di ricezione, elaborazione e trasmissione è eseguita in un unico flusso di controllo. La concorrenza è quindi stata serializzata nel ciclo principale, nel quale si decide di volta in volta quale evento servire. Questo semplifica la gestione di un database di routing condiviso, evitando problemi di sincronizzazione e race condition.

I timer non sono gestiti da thread separati, ma integrati nel ciclo di attesa tramite una `select()`, che viene chiamata con un timeout calcolato dinamicamente in base al prossimo update periodico da inviare.

2.2 Strutture dati principali

2.2.1 Messaggio RIP

La struttura del messaggio RIP segue le specifiche definite nel relativo RFC. Le strutture implementate nel programma sono le seguenti:

```
struct rip_rte
{
    uint16_t addr_family;
    uint16_t route_tag;
    uint32_t ip_address;
    uint32_t subnet_mask;
    uint32_t next_hop;
    uint32_t metric;
} __attribute__((packed));

struct rip_packet
{
    uint8_t command;
    uint8_t version;
    uint16_t must_be_zero;
    struct rip_rte entries[25];
} __attribute__((packed));
```

La struttura `rip_rte` rappresenta ogni Route Entry ed è utilizzata all'interno di `rip_packet` tramite un array con una dimensione massima di 25 elementi, come definito dalle specifiche. Per entrambe le strutture è fondamentale specificare `__attribute__((packed))` che ordina al compilatore di non inserire byte di padding per l'allineamento dei dati ottimizzato in memoria. Tale direttiva è essenziale per la serializzazione e deserializzazione del pacchetto: ricevendo un flusso di byte dalla rete, è possibile eseguire il casting con la garanzia che ogni campo sia mappato con precisione rispettando gli offset dell'header RIP. Ogni messaggio RIP è composto da un header che viene gestito tramite `rip_packet`:

Command

Campo di 8 bit che viene usato per specificare il tipo di messaggio. Il suo valore può essere:

- 1 - Request: una richiesta al destinatario del messaggio per inviare tutta o parte della sua tabella di routing. Normalmente, questo tipo di messaggi vengono inviati all'indirizzo di multicast RIP da router che si sono da poco avviati e necessitano di popolare il più velocemente possibile la loro tabella di routing. È però possibile inviare request che sono dirette ad uno specifico router (unicast). In tal caso la conseguente response non sarà inviata in multicast ma direttamente al richiedente.
- 2 - Response: un messaggio che contiene tutta o parte della tabella di routing del mittente. Tale messaggio può essere inviato in risposta ad una Request, come Unsolicited Update generato dal mittente oppure come triggered update causato da un cambiamento nella topologia della rete.

Version

Il campo viene utilizzato per indicare la versione del protocollo (1 o 2) che il mittente del messaggio sta utilizzando.

Must be zero

Due byte riempiti da una serie di 0.

RIP entry

La parte restante del datagramma UDP contiene un elenco delle Route Entry.

Ogni Route Entry è gestita tramite la struttura `rip_rte` che contiene i seguenti campi:

Address family identifier

L'AFI è il tipo di indirizzo specificato nella entry. I due valori possibili sono 0 o 2. Il 2 rappresenta IPv4, mentre 0 viene utilizzato quando il messaggio è una Request per ricevere tutta la tabella di routing del destinatario.

Route tag

Lo scopo di questo campo (non implementato nella presente realizzazione) è quello di fornire un metodo per distinguere le rotte RIP appartenenti ad un altro dominio, che può utilizzare sempre RIP, un altro IGP oppure un EGP.

IP address

Contiene l'indirizzo di rete o di un host a cui la rotta fa riferimento

Subnet mask

A differenza del RIPv1, specificando la subnet mask è possibile utilizzare RIP in reti classless.

Next hop

Contiene l'indirizzo IP a cui i pacchetti destinati alla rete identificata dall'indirizzo e subnet mask specificati sopra devono essere inoltrati. Specificando un valore di 0.0.0.0 si vuole indicare che il next-hop è il mittente del presente messaggio.

Metric

Indica la metrica del mittente per la destinazione specificata nella entry. Per i messaggi di Request, tale campo viene impostato a 16.

2.2.2 Associazione CIDR - interfaccia

```
struct network_config
{
    uint32_t network;
    uint32_t netmask;
    char interface_name[IF_NAMESIZE];
    uint32_t local_ip;
};
```

Questa struttura è il registro globale delle reti/interfacce che l'applicazione deve gestire. Il suo scopo non è "fare routing", ma fornire al resto del programma l'elenco delle reti locali abilitate, con i relativi metadati locali.

Questa struttura è la base di molte operazioni fondamentali del funzionamento dell'applicazione:

- All'avvio del programma, tale struttura viene allocata in un array ed utilizzata per caricare dal file di configurazione le informazioni relative agli indirizzi delle interfacce sulle quali il protocollo deve operare. Per ogni rete si memorizzano il suo indirizzo e la relativa netmask.
- Subito dopo il caricamento dei dati del file di configurazione, le informazioni caricate sono utilizzate dalla procedura `create_rip_socket()` per stabilire su quali interfacce effettuare il Join al gruppo multicast del RIP. Per ogni indirizzo IP valido, si aggiornano i campi `interface_name` e `local_ip` che permettono, rispettivamente, di associare la `network` alla relativa interfaccia e indirizzo IP locale dell'interfaccia a cui appartiene.
- Successivamente, i dati, ad ora completi di tutti gli attributi, vengono utilizzati per inizializzare il RIP database in modo da avviare la logica di routing.
- Nel resto del programma, l'array di tipo `network_config` viene continuamente aggiornato rispetto al funzionamento delle interfacce, in modo da gestire tutta una serie di operazioni necessarie al funzionamento del protocollo RIP. Tra

queste operazioni c'è la gestione di un eventuale link failure con la conseguente gestione dell'avvelenamento della rotta, la decisione su quali interfacce locali inviare gli unsolicited update agli altri vicini e capire da quale interfaccia è arrivato un messaggio.

2.2.3 Entry del RIP database

```
struct route_entry
{
    uint32_t network;
    uint32_t subnet_mask;
    uint32_t next_hop;
    uint32_t metric;
    char interface_name[IF_NAMESIZE];
    time_t last_update;
    time_t invalid_since;
    int is_local;
};
```

Ogni elemento inserito nel RIP database ha questo tipo di struttura, in modo da memorizzare tutti i dati necessari alla logica di routing:

Network, Subnet mask, Next hop, Metric

Questi quattro campi non necessitano di ulteriori spiegazioni, poiché hanno la stessa funzione descritta nella sottosezione 2.2.1.

Interface name

Il campo indica l'interfaccia di uscita associata alla rete in questione.

Last update

Questo campo memorizza l'istante dell'ultimo aggiornamento ricevuto per la rotta. Viene utilizzato insieme a `invalid_since` per determinare quando una rotta deve essere considerata scaduta e marcata come irraggiungibile impostando la metrica a 16. In questa applicazione, il timeout è fissato a 60 secondi, pari a tre volte l'update timer.

Invalid since

Nel momento in cui una rotta viene dichiarata come irraggiungibile, in questo campo viene salvato l'istante a partire dal quale è considerata invalida, in modo da permettere al garbage collector di rimuoverla se, per ulteriori 40 secondi, non arrivano nuovi aggiornamenti.

Is local

Questo flag serve a distinguere le rotte direttamente connesse al nodo da quelle apprese tramite RIP, in modo da escludere le prime da alcune operazioni di gestione che caratterizzano il protocollo.

2.2.4 RIP database

```
struct routing_table
{
    struct route_entry entries[MAX_ROUTING_TABLE_ENTRIES];
    int num_entries;
};
```

Ogni elemento del RIP database viene memorizzato all'interno di una struttura di tipo `routing_table`, che rappresenta la tabella di routing (RIP database) interna mantenuta dall'applicazione:

Entries

Tramite un array, è possibile memorizzare tutte le rotte note al momento. Ogni posizione della struttura ospita un elemento di tipo `route_entry`, che descrive una singola rete con i relativi metadati di routing.

Num entries

Indica semplicemente il numero di elementi attualmente presenti nella tabella in modo tale da conoscere quanti slot dell'array sono effettivamente occupati e di iterare correttamente sulle rotte valide.

3 Analisi del codice

3.1 Avvio e ciclo principale

```
int main()
{
    // ... gestione di SIGINT e SIGTERM ...

    int sock = create_rip_socket();
    int nl_sock = create_netlink_socket();

    srand(time(NULL) ^ getpid());

    init_rip_database();
    print_routing_table();

    // ... inizializzazione dei buffer ...

    while (1)
    {
        if (shutdown_requested)
            break;

        expire_timed_out_routes(sock);
        refresh_local_interface_routes(sock);

        FD_ZERO(&readfds);
        FD_SET(sock, &readfds);
        FD_SET(nl_sock, &readfds);

        if (should_send_update(...))
        {
            send_unsolicited_update(sock);
        }

        select(...);

        if (activity > 0 && FD_ISSET(sock, &readfds))
        {
            recvfrom(...);
            process_rip_packet(...);
        }
        else if (activity > 0 && FD_ISSET(nl_sock, &readfds))
        {
            handle_netlink_link_events(nl_sock);
        }
    }

    graceful_shutdown(sock);
}
```

Il brano di codice sopra riportato è una versione ridotta del codice esistente che mostra soltanto la struttura principale dell'avvio e del loop principale, anche utilizzando nomi di metodi e funzioni che in realtà non esistono. Di seguito vengono riportate le sezioni di codice con la relativa descrizione:

```
• // ... gestione di SIGINT e SIGTERM ...

int sock = create_rip_socket();
int nl_sock = create_netlink_socket();

srand(time(NULL) ^ getpid());

init_rip_database();

// ... inizializzazione dei buffer ...
```

In questa prima parte di codice viene chiamata la funzione `create_rip_socket()` che si occupa di caricare la configurazione dal relativo file, creare il socket idoneo al traffico UDP di RIP ed effettuare il bind sulla porta 520. Successivamente, parte un loop che scorre tutte le interfacce del sistema e, per ognuna, effettua l'iscrizione al gruppo multicast del RIP solo se l'interfaccia non è di loopback, ha un indirizzo IPv4, è attiva (ha il link attivo) e il suo indirizzo IP rientra nelle reti configurate tramite file. Per terminare, viene disabilitato il loopback dei pacchetti multicast (`IP_MULTICAST_LOOP = 0`) tramite `setsockopt`. In questo modo, i messaggi RIP inviati in multicast dall'host non verranno ricevuti dallo stesso socket. Successivamente, viene creato il socket netlink che permette all'applicazione di interagire con il kernel aggiungendo e modificando la tabella di routing dell'host. Poi, si procede a inizializzare il generatore di numeri casuali usato da `rand()` garantendo che il seme generato sia il più unico e imprevedibile possibile. In questo caso, ad ogni avvio del programma `time(NULL)` è sempre diverso, `getpid()` cambia tra processi diversi e facendo lo XOR tra questi due si ottiene un seed diverso ad ogni avvio in modo che la sequenza dei valori casuali usati per la generazione dei jitter del timer tra i vari update sia sempre diversa. In seguito, si procede ad effettuare la prima inizializzazione del RIP database, semplicemente aggiungendo le rotte relative alle interfacce locali.

```
• // ...

while (1)
{
    if (shutdown_requested)
        break;

    expire_timed_out_routes(sock);
    refresh_local_interface_routes(sock);

    // ...
```

Ora inizia il loop vero e proprio che "dà vita" all'intera applicazione. Il primo controllo che viene effettuato è quello per determinare se è stato inviato un segnale di terminazione forzata dell'applicazione (ad esempio con un Ctrl + C oppure, se il programma si trova all'interno di un container, con uno stop forzato del container). La variabile `shutdown_requested` è inizializzata con valore 0. Quando un evento di terminazione viene catturato, il suo valore passa a 1, in modo da rendere vero il controllo. Successivamente, vengono chiamate due funzioni che

permettono di gestire le rotte nella tabella di routing. La prima, successivamente descritta nel dettaglio, permette di eliminare dalla tabella di routing le rotte contrassegnate come irraggiungibili dopo il trascorrere del Garbage Collection Timer, ma anche contrassegnare come irraggiungibili le rotte delle quali non si sono ricevuti update dopo un tempo pari al Route Timeout. La seconda funzione chiamata permette di aggiornare lo stato delle interfacce locali in modo da aggiornare di conseguenza la tabella di routing.

```
• // ...

FD_ZERO(&readfds);
FD_SET(sock, &readfds);
FD_SET(nl_sock, &readfds);

if (should_send_update(...))
{
    send_unsolicited_update(sock);
}

select(...);

if (activity > 0 && FD_ISSET(sock, &readfds))
{
    recvfrom(...);
    process_rip_packet(...);
}
else if (activity > 0 && FD_ISSET(nl_sock, &readfds))
{
    handle_netlink_link_events(nl_sock);
}

graceful_shutdown(sock);
}
```

Questo blocco prepara e gestisce l'I/O multiplexing (tramite la chiamata di sistema `select`) per monitorare due socket diversi in un ciclo di eventi. Con le prime tre istruzioni si inizializza e azzerava il set di descrittori di file `readfds` e con le due `FD_SET` si aggiunge il socket principale UDP e il socket netlink al set di monitoraggio in lettura. Con il controllo successivo viene determinato se è il momento di inviare un Unsolicited Update su tutte le interfacce attive tramite una semplice differenza tra il timestamp del momento dell'effettuazione del controllo e quello memorizzato poco prima dell'invio precedente. Infine, viene utilizzata la `select` per bloccare l'esecuzione del programma finché non arrivano messaggi su uno dei due socket creati oppure scade il timeout. Il timeout viene ricalcolato ad ogni iterazione in base a quanto tempo manca al next unsolicited update. In questo modo, il timeout scatterà nell'istante esatto in cui è necessario inviare un update in modo da far procedere il loop e permettere tale invio. Se invece la `select` rileva dell'attività su uno dei due socket, il programma procede e tramite i successivi controlli viene stabilito quale dei due socket ha ricevuto un messaggio, gestendo con le relative funzioni ognuno dei due casi. Per terminare, se il programma viene terminato forzatamente (quindi il loop viene interrotto con

un break), viene chiamata la funzione `graceful_shutdown` che non fa altro che impostare tutte le entry della tabella di routing come irraggiungibili e inviare un update a tutti i vicini. In questo modo se un nodo viene spento all'improvviso gli altri vicini possono velocemente ricalcolare i next hop.

3.2 Caricamento della configurazione da file

3.2.1 `load_config()`

```
static void load_config()
{
    FILE *f = fopen(CONFIG_FILE, "r");
    if (!f)
    {
        fprintf(stderr, "Warning: Could not open config file %s\n",
            CONFIG_FILE);
        return;
    }

    char line[256];
    while (fgets(line, sizeof(line), f) && num_networks < MAX_NETWORKS)
    {
        line[strcspn(line, "\n")] = 0;

        if (strncmp(line, "NET=", 4) == 0)
        {
            parse_cidr(line + 4, &networks[num_networks].network, &
                networks[num_networks].netmask);
            printf("Loaded network: %s\n", line + 4);
            num_networks++;
        }
    }
    fclose(f);
}
```

Questa funzione viene chiamata da `create_rip_socket` e il suo compito è quello di leggere il file di configurazione (che deve essere rinominato come "router.conf"), del quale di seguito viene proposto un esempio:

```
NET=10.12.1.0/24
NET=10.12.10.0/24
NET=10.12.4.0/24
NET=10.12.5.0/24
```

In questo tipo di file vengono specificate tutte le reti in cui il processo RIP deve essere attivato. La routine che si occupa di leggere questo file non fa altro che aprirlo in lettura e successivamente iterare tra le righe del file fino alla sua fine oppure fino a quando il numero di righe lette supera il limite massimo di 10 rotte. Tramite la funzione `parse_cidr` si carica il valore del CIDR specificato nel file all'interno di `networks`, una variabile di tipo `network_config` fondamentale per il funzionamento dell'intero programma.

3.3 Gestione delle rotte

3.3.1 process_rip_packet()

```
void process_rip_packet(int sock, struct rip_packet *pkt, int
bytes_received, struct sockaddr_in *sender) {
    int entries = (bytes_received - 4) / sizeof(struct rip_rte);
    if (entries > 25) entries = 25;
    if (entries <= 0) return;

    // Risoluzione IP e interfaccia di ricezione del mittente
    char sender_ip[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &(sender->sin_addr), sender_ip, INET_ADDRSTRLEN);
    char *iface = get_interface_for_ip(sender->sin_addr.s_addr);

    /* --- RIP REQUEST (Command 1) --- */
    if (pkt->command == 1) {
        // Richiesta intera tabella (Family=0, Metric=16)
        if (entries == 1 && ntohs(pkt->entries[0].addr_family) == 0 &&
            ntohl(pkt->entries[0].metric) == 16) {
            send_full_table_unicast(sock, sender, iface);
            return;
        }

        // Richiesta per rotte specifiche: compila le metriche e rispondi
        via unicast
        for (int i = 0; i < entries; i++) {
            struct route_entry *r = find_route(pkt->entries[i].ip_address
, pkt->entries[i].subnet_mask);
            pkt->entries[i].metric = htonl(r ? r->metric : 16);
        }
        pkt->command = 2; // Trasforma il pacchetto in RIP Response
        sendto(sock, pkt, 4 + entries * sizeof(struct rip_rte), 0, (
struct sockaddr *)sender, sizeof(*sender));
    }
    /* --- RIP RESPONSE / UPDATE (Command 2) --- */
    else if (pkt->command == 2) {
        int changed = 0;
        for (int i = 0; i < entries; i++) {
            struct rip_rte *e = &pkt->entries[i];
            uint32_t metric = ntohl(e->metric);

            // Validazione AF_INET (2) e range metrica [1, 16]
            if (ntohs(e->addr_family) == 2 && metric > 0 && metric <= 16)
            {
                if (process_route(e->ip_address, e->subnet_mask, e->
next_hop, metric, sender_ip, iface))
                    changed = 1;
            }
        }
        // Triggered Update immediato in caso di mutamento topologico
        if (changed) send_unsolicited_update(sock);
    }
}
```

Lo scopo di questa funzione è quello di ricevere e processare tutti i pacchetti RIP che vengono ricevuti. La sua struttura è suddividibile in tre sezioni principali: il calcolo del numero delle rotte (nel caso sia stato ricevuto un Update), la gestione delle Request e la gestione degli Update. Partendo dai parametri, sono passati rispettivamente il descrittore del socket UDP, il puntatore alla struttura che contiene il payload RIP ricevuto (intestazione RIP + voci della tabella di routing), il numero effettivo di byte letti dal socket e il puntatore alla struttura che contiene l'indirizzo IP e la porta del mittente. La prima sezione della routine non fa altro che determinare quante entry contiene il pacchetto ricevuto. In pratica, dalla dimensione effettiva del pacchetto ricevuto, si sottraggono 4 byte (la dimensione dell'header) e quindi si ottiene la dimensione effettiva che le entry occupano all'interno del pacchetto. A questo punto si divide per la dimensione di ogni entry per ottenere il loro numero. Nel caso superino il valore massimo prefissato di 25, si considerano solo le prime 25 entry. Contestualmente, vengono ricavati l'indirizzo IP del mittente in formato stringa e l'interfaccia di ricezione locale (iface) tramite la funzione `get_interface_for_ip`. Successivamente, se il pacchetto è una Request (`pkt->command == 1`) si controlla che il numero di entry richieste sia maggiore di 0, sennò sarebbe una richiesta vuota. Nel caso in cui ci sia una sola entry, Address Family è impostato a 0 e la metrica dell'unica entry è impostata a 16, seguendo le regole specificate in RFC 2453 indicano che si tratta di una richiesta all'intera tabella di routing. In tal caso, si procede ad inviare l'intera tabella di routing, in modalità unicast, al richiedente. Nel caso di Request specifica per determinate rotte, si procede a iterare sulle entry e per ognuna non si fa altro che prendere il pacchetto ricevuto e modificare la metrica presente con il valore del ricevente. Successivamente si cambia il valore del campo Command, inserendo il valore 2, e infine lo si invia sempre in Unicast al mittente. Per quanto riguarda l'ultima sezione, il processamento degli Update, in questo caso si procede a iterare su tutte le rotte contenute nel pacchetto e per ognuna valida (Address Family è impostato 2) si procede a passare alla funzione di gestione della logica RIP (`process_route()`) tutte le informazioni necessarie a prendere la decisione corretta sull'aggiornamento della tabella di routing. Se almeno una rotta nella tabella viene aggiornata, si procede ad inviare un Unsolicited Update a tutti i vicini in modo da evitare tempi di convergenza troppo elevati.

3.3.2 process_route()

```
int process_route(uint32_t net, uint32_t mask, uint32_t pkt_nh, uint32_t
rcv_metric, const char *sender_ip, const char *sender_iface)
{
    /* --- PARAMETRI ROTTA --- */
    uint32_t metric = (rcv_metric + 1 > 16) ? 16 : rcv_metric + 1;
    uint32_t next_hop = (pkt_nh == 0 || pkt_nh == inet_addr("0.0.0.0")) ?
        inet_addr(sender_ip) : pkt_nh;
    const char *iface = (sender_iface && sender_iface[0] != '\0') ?
        sender_iface : "RIP";

    struct route_entry *rt = find_route(net, mask);

    // 1. Nuova rotta raggiungibile
    if (!rt && metric < 16) {
        add_route(net, mask, next_hop, metric, iface, 0);
        struct route_entry *new_rt = find_route(net, mask);
        if (new_rt) add_or_replace_kernel_route(new_rt);
        return 1;
    }

    if (!rt) return 0;

    // Se una rotta locale e' stata avvelenata (link down), permettiamo
    // il fallback a dinamica
    if (rt->is_local && rt->metric >= 16 && metric < 16) {
        rt->is_local = 0;
        strncpy(rt->interface_name, iface, IF_NAMESIZE - 1);
        rt->interface_name[IF_NAMESIZE - 1] = '\0';
    }

    // 2. Aggiornamento dallo stesso next-hop (anche se la metrica
    // peggiora)
    if (rt->next_hop == next_hop) {
        int changed = (rt->metric != metric);
        rt->metric = metric;
        rt->last_update = time(NULL);

        if (metric == 16) {
            delete_kernel_route(rt);
            rt->invalid_since = time(NULL);
        } else {
            rt->invalid_since = 0;
            if (changed) add_or_replace_kernel_route(rt);
        }
        return changed;
    }

    // 3. Percorso migliore offerto da un router alternativo
    if (metric < rt->metric) {
        rt->next_hop = next_hop;
        rt->metric = metric;
        rt->last_update = time(NULL);
        rt->invalid_since = 0;
        strncpy(rt->interface_name, iface, IF_NAMESIZE - 1);
        rt->interface_name[IF_NAMESIZE - 1] = '\0';
    }
}
```

```
        add_or_replace_kernel_route(rt);  
        return 1;  
    }  
  
    return 0; // Rotta alternativa con costo uguale o peggiore: ignorata  
}
```

Come accennato nella descrizione della funzione precedente, questa funzione è responsabile di prendere le decisioni riguardo alle operazioni di eventuale modifica della tabella di routing a seguito della ricezione di un Update. Per quanto riguarda i parametri, ritengo che non necessitino di spiegazioni in quanto hanno tutti nomi auto-descrittivi. Le prime operazioni servono a determinare la nuova metrica e il rispettivo next hop per la rotta in questione. Come da standard, la metrica viene incrementata di 1, mentre se il next hop è impostato a 0.0.0.0 è implicito che l'indirizzo sia quello del mittente, altrimenti si utilizza quello specificato. Proseguendo, se la rotta non è presente nel rip database e se la metrica aggiornata è inferiore 16, si procede a passarla alla routine che si occupa di modificare direttamente il database e la tabella di routing del kernel. Nel caso in cui, invece, la rotta sia già presente in memoria, si procede a gestire il caso speciale di fallback dinamico se la rotta era una rotta locale precedentemente avvelenata a causa di un link down. In questo scenario la rotta locale ha metrica 16, ma un vicino la annuncia con una metrica valida, di conseguenza si procede a effettuare la conversione a rotta dinamica RIP. Questa operazione non rischia di generare loop di routing, in quanto lo split horizon (adottato in questa implementazione) impedisce l'effetto "eco" delle rotte. Successivamente, la decisione di aggiornamento segue le specifiche standard RFC:

Il next hop corrisponde a quello attualmente presente nel database

In questo caso, la metrica viene sempre aggiornata, anche se peggiora. Se la nuova metrica è 16 si procede ad eliminare la rotta dalla tabella del kernel. Nel database la rotta rimane installata, ma viene considerata irraggiungibile.

Il next hop non corrisponde a quello attualmente presente nel database

Se la rotta proviene da un router diverso da quello che l'ha precedentemente diffusa, si effettuano controlli sulla metrica: se è peggiore o uguale si ignora, se è migliore si modifica il database e la tabella del kernel con i rispettivi nuovi valori.

3.3.3 add_route()

```
void add_route(uint32_t net, uint32_t mask, uint32_t nh, uint32_t metric,
               const char *iface, int is_local)
{
    if (rip_database.num_entries >= MAX_ROUTING_TABLE_ENTRIES) return;

    struct route_entry *rt = &rip_database.entries[rip_database.
num_entries++];
    rt->network = net;
    rt->subnet_mask = mask;
    rt->next_hop = nh;
    rt->metric = metric;
    rt->is_local = is_local;
    rt->last_update = time(NULL);
    rt->invalid_since = 0;
    strncpy(rt->interface_name, iface, IF_NAMESIZE - 1);
}
```

Questa funzione è responsabile delle operazioni sul rip database. I parametri in input non necessitano di ulteriori spiegazioni in quanto sono autoesplicativi. La sua funzione è simile ad un comando di aggiunta di rotta statica in un router, in quanto prende in input i vari parametri ed esegue le operazioni in tabella. Il primo controllo effettuato verifica che il database non sia già pieno (contiene già 25 rotte). Successivamente, si memorizza il puntatore alla prima entry libera dell'array e si procede a popolare i campi con i rispettivi valori e incrementare il contatore delle rotte totali nel database.

3.3.4 find_route()

```
struct route_entry *find_route(uint32_t network, uint32_t subnet_mask)
{
    for (int i = 0; i < rip_database.num_entries; i++)
    {
        // Se IP di rete e Subnet Mask coincidono, abbiamo trovato la
        rotta
        if (rip_database.entries[i].network == network &&
            rip_database.entries[i].subnet_mask == subnet_mask)
        {
            return &rip_database.entries[i];
        }
    }
    return NULL;
}
```

Questa funzione è fondamentale per centralizzare il loop di ricerca delle rotte nel database e facilitare le operazioni che necessitano di ricercare una specifica rotta. Prende in input l'indirizzo di rete e la netmask e itera su tutte le entry del database fino a trovare il rispettivo indice. Se la rotta viene trovata, si restituisce il puntatore all'elemento route_entry, altrimenti NULL.

3.3.5 send_routing_table()

```
static void send_routing_table(int sock, struct sockaddr_in *dest, const
    char *split_horizon_iface, struct in_addr *out_iface_ip)
{
    if (rip_database.num_entries == 0) return;

    struct rip_packet pkt = { .command = 2, .version = 2 };
    int count = 0;

    for (int i = 0; i < rip_database.num_entries; i++)
    {
        struct route_entry *route = &rip_database.entries[i];
        uint32_t metric = route->metric;

        // Split Horizon con Poisoned Reverse
        if (!route->is_local && strcmp(route->interface_name,
            split_horizon_iface) == 0) {
            metric = 16;
        }

        pkt.entries[count++] = (struct rip_rte){
            .addr_family = htons(2),
            .ip_address = route->network,
            .subnet_mask = route->subnet_mask,
            .next_hop = 0, // 0.0.0.0
            .metric = htonl(metric)
        };

        // Frammentazione: invio e reset ogni 25 route (RFC 2453)
        if (count == 25) {
            send_rip_packet(sock, &pkt, 25, dest, out_iface_ip);
            memset(&pkt, 0, sizeof(struct rip_packet));
            pkt.command = 2; pkt.version = 2;
            count = 0;
        }

        // Invio dell'eventuale pacchetto parziale residuo
        if (count > 0) {
            send_rip_packet(sock, &pkt, count, dest, out_iface_ip);
        }
    }
}
```

Questa funzione permette di inviare le informazioni contenute nel RIP database sul socket passato al momento della chiamata. Tra i parametri più importanti c'è sicuramente `split_horizon_iface` che identifica l'interfaccia fisica di uscita su cui trasmettere l'annuncio. La sua importanza è legata alla risoluzione dei loop di routing e del count to infinity. In pratica, Split Horizon è un meccanismo che impedisce ad un router di inviare informazioni di route al router da cui le ha apprese. Nella modalità con poisoned reverse (implementata in questa applicazione), il router, anziché omettere le rotte apprese da una specifica interfaccia, le include comunque nell'aggiornamento inviato su tale interfaccia, ma impostando la metrica a 16. Nelle prime righe di codice vengono inizializzati i campi base del pacchetto di Update e successivamente si avvia un loop che itera su tutte le entry del database. Per ognuna, si controlla se l'interfaccia

corrisponde a quella indicata nel parametro `split_horizon_iface` e in tal caso la sua metrica annunciata viene impostata 16. In tutti gli altri casi si procede alla forgiatura dell'Update con tutti i relativi parametri. Se il numero di rotte supera il limite massimo di 25 entry, si procede ad inviare un primo pacchetto completo e subito dopo un altro con le restanti rotte.

3.4 Timer e gestione del database

3.4.1 `expire_timed_out_routes()`

```
int expire_timed_out_routes(int sock)
{
    time_t now = time(NULL);
    int table_changed = 0;

    // 1. Garbage Collection: rimozione definitiva rotte con metrica 16
    // scadute
    for (int i = rip_database.num_entries - 1; i >= 0; i--)
    {
        struct route_entry *route = &rip_database.entries[i];

        if (route->is_local || route->metric < 16 || route->invalid_since
            == 0)
            continue;

        if (difftime(now, route->invalid_since) >=
            GARBAGE_COLLECTION_TIMER)
        {
            delete_kernel_route(route);
            remove_route_at_index(i);
            table_changed = 1;
        }
    }

    // 2. Timeout: invalidazione e avvelenamento (metrica = 16) rotte non
    // aggiornate
    for (int i = 0; i < rip_database.num_entries; i++)
    {
        struct route_entry *route = &rip_database.entries[i];

        if (route->is_local || route->metric >= 16)
            continue;

        if (difftime(now, route->last_update) >= ROUTE_TIMEOUT)
        {
            route->metric = 16;
            route->invalid_since = now;
            delete_kernel_route(route);
            table_changed = 1;
        }
    }

    // 3. Triggered update se lo stato della tabella e' mutato
    if (table_changed)
```

```
    {  
        send_unsolicited_update(sock);  
    }  
  
    return table_changed;  
}
```

Questa funzione viene chiamata nel `main()` ed è responsabile della rimozione delle rotte che sono state contrassegnate come irraggiungibili (metrica a 16) e che hanno superato il tempo massimo di permanenza nel database. Inoltre, si occupa anche di impostare la metrica a 16 per le rotte delle quali non è stato ricevuto nessun update entro lo specifico timeout. La prima cosa che viene fatta è memorizzare l'istante in cui la funzione viene chiamata, in modo da effettuare le opportune differenze tra i timestamp. Con il primo ciclo si itera su tutte le rotte nel database in cerca di quelle in cui la differenza tra il timestamp attuale e il parametro `invalid_since` supera il timer per le operazioni di garbage collection. Per queste ultime si procede all'eliminazione dal database. La seconda parte della funzione si occupa di iterare su tutte le rotte e ricercare quelle che non hanno ricevuto nessun update nel tempo valido, impostando la loro metrica a 16. Infine, per qualsiasi modifica venga effettuata, si procede ad inviare un update a tutti i vicini.

3.4.2 refresh_local_interface_routes()

```
int refresh_local_interface_routes(int sock)  
{  
    int table_changed = 0;  
    time_t now = time(NULL);  
  
    for (int iface_idx = 0; iface_idx < num_networks; iface_idx++)  
    {  
        if (networks[iface_idx].interface_name[0] == '\0')  
            continue;  
  
        struct in_addr current_ip;  
        int iface_up = interface_is_up(networks[iface_idx].interface_name,  
            &current_ip);  
        struct route_entry *route = find_route(networks[iface_idx].  
            network, networks[iface_idx].netmask);  
  
        // Caso 1: L'interfaccia e' attiva (UP)  
        if (iface_up)  
        {  
            networks[iface_idx].local_ip = current_ip.s_addr;  
  
            // Ripristina preferenza locale se la rete era stata appresa  
            dinamicamente (fallback)  
            if (route != NULL && !route->is_local)  
            {  
                route->next_hop = inet_addr("0.0.0.0");  
                route->metric = 1;  
                route->is_local = 1;  
                route->invalid_since = 0;  
                route->last_update = now;  
            }  
        }  
    }  
}
```

```
        strncpy(route->interface_name, networks[iface_idx].
interface_name, IF_NAMESIZE - 1);
        add_or_replace_kernel_route(route);
        table_changed = 1;
    }

    // Ripristina la rotta directly connected se precedentemente
    avvelenata/invalida
    if (route != NULL && route->is_local && (route->metric != 1
|| route->invalid_since != 0))
    {
        route->metric = 1;
        route->invalid_since = 0;
        route->last_update = now;
        add_or_replace_kernel_route(route);
        table_changed = 1;
    }
}
// Caso 2: L'interfaccia e' inattiva (DOWN)
else
{
    // Avvelena (metrica = 16) le rotte dinamiche che usavano
    questa subnet come next-hop
    for (int route_idx = 0; route_idx < rip_database.num_entries;
route_idx++)
    {
        struct route_entry *dep = &rip_database.entries[route_idx
];

        if (dep->is_local || dep->metric >= 16)
            continue;

        if (ip_in_network(dep->next_hop, networks[iface_idx].
network, networks[iface_idx].netmask))
        {
            delete_kernel_route(dep);
            dep->metric = 16;
            dep->invalid_since = now;
            dep->last_update = now;
            table_changed = 1;
        }
    }

    // Avvelena la rotta directly connected
    if (route != NULL && route->is_local && route->metric != 16)
    {
        route->metric = 16;
        route->invalid_since = now;
        route->last_update = now;
        delete_kernel_route(route);
        table_changed = 1;
    }
}

if (table_changed)
{
```

```
        send_unsolicited_update(sock);  
    }  
  
    return table_changed;  
}
```

Questa funzione presenta una logica articolata in quanto gestisce le operazioni sul database nel momento in cui un'interfaccia locale cambia stato. L'unico parametro accettato in input è il descrittore del socket. L'esecuzione inizia salvando il timestamp di chiamata e iterando su tutte le rotte nel database alle quali è associata un'interfaccia di uscita con un nome valido. Per ognuna di esse controlla lo stato tramite la funzione helper `interface_is_up` (la quale verifica la presenza del carrier fisico, l'assenza del flag di loopback e, in caso di successo, estrae l'indirizzo IPv4 corrente). Il flusso operativo si dirama in due scenari:

Interfaccia up

Si procede a copiare il suo indirizzo nell'area di memoria puntata dal secondo parametro. A questo punto si gestiscono due possibili condizioni di ripristino:

- **Ripristino da fallback dinamico:** se la rete locale era stata precedentemente appresa via RIP da un router vicino (diventando una rotta remota con `is_local == 0`) a causa di un prolungato disservizio del link, non appena l'interfaccia torna attiva le viene riassegnata priorità assoluta. La voce viene convertita nuovamente in rotta locale (`is_local = 1`), con next-hop 0.0.0.0, metrica 1 e reinserimento nel kernel.
- **Ripristino di rotta locale avvelenata:** se la rotta era già marcata come locale ma risultava precedentemente dismessa o avvelenata (`metric != 1` o `invalid_since != 0`), viene ripristinata la metrica nominale a 1, azzerato il timer di invalidazione e reinstallata la rotta nel kernel.

Interfaccia down

Nel caso in cui invece il suo stato sia down, si procede immediatamente ad effettuare le seguenti operazioni:

- **Avvelenamento rotte dipendenti:** tutte le rotte dinamiche presenti nel database il cui `next_hop` appartiene alla subnet dell'interfaccia disconnessa diventano irraggiungibili; vengono eliminate dal kernel e la loro metrica RIP è impostata a 16 (*poisoning*), avviando il timer `invalid_since`.
- **Avvelenamento rotta locale:** la rotta direttamente associata all'interfaccia viene rimossa dalla tabella del kernel e contrassegnata con metrica 16.

3.4.3 graceful_shutdown()

```
void graceful_shutdown(int sock)
{
    if (rip_database.num_entries == 0)
        return;

    time_t now = time(NULL);

    // 1. Avvelena tutte le rotte locali impostando la metrica a 16 (
    // infinita)
    for (int i = 0; i < rip_database.num_entries; i++)
    {
        struct route_entry *route = &rip_database.entries[i];
        if (route->is_local)
        {
            route->metric = 16;
            route->invalid_since = now;
            route->last_update = now;
        }
    }

    // 2. Notifica immediata ai router vicini della disconnessione
    send_unsolicited_update(sock);

    // 3. Ripristino dello stato: rimozione rotte dal kernel e
    // svuotamento database
    for (int i = rip_database.num_entries - 1; i >= 0; i--)
    {
        delete_kernel_route(&rip_database.entries[i]);
        if (!rip_database.entries[i].is_local)
        {
            delete_kernel_route(&rip_database.entries[i]);
        }
        remove_route_at_index(i);
    }
}
```

Il ruolo di questa funzione è quello di procedere ad avvelenare tutte le rotte installate e inviare un update a tutti i vicini nel momento in cui viene catturato un segnale di interruzione. L'applicazione intercetta i segnali POSIX SIGINT (inviato da tastiera con Ctrl+C) e SIGTERM (segnale standard di terminazione inviato da comandi come docker stop o kill) e, invece di terminare immediatamente il processo in modo improvviso (lasciando le rotte nel kernel e costringendo i vicini ad attendere il timeout della rotta per riconvergere), il gestore imposta un flag di terminazione consentendo al loop principale nel main di uscire in modo controllato ed invocare graceful_shutdown(sock), che provvede ad avvelenare le rotte verso i vicini e a rimuovere le voci inserite nella tabella di routing del kernel prima della terminazione definitiva. Una particolare attenzione va al controllo if (!rip_database.entries[i].is_local) in quanto permette di non eliminare dal kernel le rotte delle interfacce locali. Se questo non accadesse, allo spegnimento dell'applicazione l'host non sarebbe più in grado di determinare quale interfaccia utilizzare per comunicare in subnet locali e quindi non potrebbe più comunicare con i dispositivi appartenenti ai segmenti di rete direttamente connessi

3.5 Integrazione con il kernel

3.5.1 netlink_send_route_request()

```
static int netlink_send_route_request(int sock, int cmd, int flags,
                                     uint32_t dst, uint8_t dst_prefix_len, uint32_t gw, int has_gw, int
                                     if_idx, uint32_t metric)
{
    struct {
        struct nlmsgghdr n;
        struct rtmsg r;
        char buf[4096];
    } req;

    memset(&req, 0, sizeof(req));
    // Header netlink di base.
    req.n.nlmsg_len = NLMSG_LENGTH(sizeof(struct rtmsg));
    req.n.nlmsg_flags = NLM_F_REQUEST | NLM_F_ACK | flags;
    req.n.nlmsg_type = cmd;
    req.n.nlmsg_seq = (uint32_t)time(NULL);

    req.r.rtm_family = AF_INET;
    req.r.rtm_table = RT_TABLE_MAIN;
    req.r.rtm_scope = has_gw ? RT_SCOPE_UNIVERSE : RT_SCOPE_LINK;
    req.r.rtm_dst_len = dst_prefix_len;

    // Per le nuove rotte si imposta anche protocollo e tipo.
    if (cmd != RTM_DELROUTE) {
        req.r.rtm_protocol = RTPROT_STATIC;
        req.r.rtm_type = RTN_UNICAST;
    }

    // Aggiunta degli attributi Netlink (RTA)
    if (has_gw && netlink_add_attribute(&req.n, sizeof(req), RTA_GATEWAY,
        &gw, sizeof(gw)) < 0) return -1;
    if (netlink_add_attribute(&req.n, sizeof(req), RTA_DST, &dst, sizeof(
        dst)) < 0) return -1;
    if (if_idx > 0 && netlink_add_attribute(&req.n, sizeof(req), RTA_OIF,
        &if_idx, sizeof(if_idx)) < 0) return -1;
    if (cmd != RTM_DELROUTE && metric > 0 &&
        netlink_add_attribute(&req.n, sizeof(req), RTA_PRIORITY, &metric,
        sizeof(metric)) < 0) return -1;

    struct sockaddr_nl kaddr = { .nl_family = AF_NETLINK };
    struct iovec iov = { .iov_base = &req.n, .iov_len = req.n.nlmsg_len
    };
    struct msghdr msg = { .msg_name = &kaddr, .msg_namelen = sizeof(kaddr
    ), .msg_iov = &iov, .msg_iovlen = 1 };

    if (sendmsg(sock, &msg, 0) < 0) return -1;

    // Ricezione e parsing dell'ACK / errore dal Kernel
    unsigned char reply[4096];
    struct iovec r_iov = { .iov_base = reply, .iov_len = sizeof(reply) };
    struct msghdr r_msg = { .msg_name = &kaddr, .msg_namelen = sizeof(
        kaddr), .msg_iov = &r_iov, .msg_iovlen = 1 };
```

```
ssize_t len = recvmsg(sock, &r_msg, 0);
if (len < 0) return -1;

for (struct nlmsg_hdr *nlh = (struct nlmsg_hdr *)reply; NLMSG_OK(nlh, (
unsigned int)len); nlh = NLMSG_NEXT(nlh, len)) {
    if (nlh->nlmsg_type == NLMSG_ERROR) {
        struct nlmsgerr *err = (struct nlmsgerr *)NLMSG_DATA(nlh);
        if (err->error == 0 || (cmd == RTM_DELROUTE && -err->error ==
ESRCH))
            return 0; // Successo o eliminazione idempotente rotta
            inesistente
        return -1;
    }
    if (nlh->nlmsg_type == NLMSG_DONE) return 0;
}
return 0;
}
```

Questa funzione permette di gestire l'interazione a basso livello con il sottosistema di rete del kernel tramite User-Kernel space communication con il socket Netlink. Il suo compito è formattare, inviare e validare le richieste di inserimento, aggiornamento o cancellazione delle rotte di sistema. I parametri più significativi in ingresso definiscono:

- `cmd`: il comando operativo (`RTM_NEWROUTE` per installare/modificare una rotta o `RTM_DELROUTE` per rimuoverla);
- `flags`: flag Netlink aggiuntivi (es. `NLM_F_CREATE` e `NLM_F_REPLACE`);
- `dst, dst_prefix_len`: indirizzo di rete di destinazione in formato binario e relativa lunghezza del prefisso CIDR;
- `gw, has_gw`: indirizzo IP del gateway/next-hop e flag booleano che ne attesta la presenza;
- `if_idx`: indice di sistema dell'interfaccia di rete di uscita;
- `metric`: metrica di instradamento (*priority/cost*) associata alla rotta nel kernel.

L'esecuzione si articola in tre fasi:

Costruzione del messaggio Netlink

Si inizia allocando e azzerando una struttura contenente l'intestazione Netlink standard (`struct nlmsg_hdr`) e il payload specifico per il routing (`struct rtmsg`). L'intestazione viene configurata con i flag `NLM_F_REQUEST` e `NLM_F_ACK`, quest'ultimo fondamentale per richiedere esplicitamente al kernel un messaggio di riscontro (positivo o errore). La struttura `rtmsg` specifica la famiglia IPv4 (`AF_INET`), la tabella principale di instradamento (`RT_TABLE_MAIN`) e la lunghezza del prefisso. Se la rotta include un gateway, il suo scope viene impostato a globale (`RT_SCOPE_UNIVERSE`), altrimenti a livello di collegamento locale (`RT_SCOPE_LINK`). Tramite la funzione helper `netlink_add_attribute`, vengono accodati al payload i Routing Attributes (RTA): destinazione (`RTA_DST`), gateway (`RTA_GATEWAY`), interfaccia di uscita (`RTA_OIF`) e metrica (`RTA_PRIORITY`).

Trasmissione al Kernel

Il buffer preparato viene incapsulato in una struttura `struct msghdr` indirizzata a `sockaddr_nl` (con indirizzo nullo per indicare il kernel) e spedito mediante la chiamata di sistema `sendmsg`.

Ricezione e validazione dell'ACK

Il demone attende in modo bloccante la risposta del kernel tramite `recvmsg`. Una volta ricevuto il riscontro, si scorrono i messaggi con `NLMSG_OK`: in presenza di un messaggio di tipo `NLMSG_ERROR`, si estrae la struttura `nlmsgerr`. Un valore `err->error == 0` indica che l'operazione è andata a buon fine. Se l'operazione richiesta è una cancellazione (`RTM_DELROUTE`) e il kernel restituisce l'errore `-ESRCH` (*No such process/route*), l'errore viene intenzionalmente ignorato. Ciò consente cancellazioni sicure e idempotenti (durante le fasi di chiusura) senza generare falsi allarmi.

3.5.2 `netlink_update_route()`

```
static int netlink_update_route(int nlmsg_type, uint32_t network,
                                uint32_t subnet_mask, uint32_t gateway, uint32_t metric, const char *
                                interface_name, int use_gateway)
{
    if (routing_netlink_sock < 0) return -1;

    int flags = (nlmsg_type == RTM_NEWROUTE) ? (NLM_F_CREATE |
        NLM_F_REPLACE) : 0;
    int if_idx = 0;

    if (interface_name && interface_name[0] != '\0') {
        if_idx = (int)if_nametoindex(interface_name);
        if (if_idx == 0) return -1;
    }

    uint8_t prefix = (uint8_t)mask_to_prefix(subnet_mask);
    return netlink_send_route_request(routing_netlink_sock, nlmsg_type,
        flags, network, prefix, gateway, use_gateway, if_idx, metric);
}
```

Questa funzione wrapper offre un'interfaccia ad alto livello per semplificare le operazioni di inserimento, aggiornamento o rimozione delle rotte nel kernel. In parole semplici, il suo scopo è quello di verificare i parametri forniti e "tradurli" nel formato richiesto da `netlink_send_route_request`. Il parametro in ingresso che governa il funzionamento della funzione è `nlmsg_type`. In pratica, qua viene specificata l'operazione Netlink desiderata: (`RTM_NEWROUTE` per creare/sostituire o `RTM_DELROUTE` per eliminare). Il flusso prosegue con una serie di passaggi:

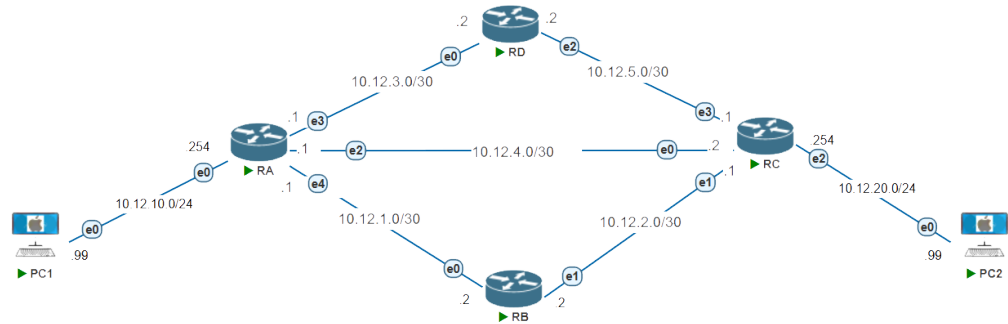
1. **Controllo del Socket:** verifica che il socket Netlink globale (`routing_netlink_sock`) sia stato correttamente inizializzato all'avvio del demone.
2. **Configurazione dei flag:** in caso di aggiunta/modifica (`RTM_NEWROUTE`), imposta la combinazione di flag `NLM_F_CREATE | NLM_F_REPLACE`, consentendo di crea-

re una nuova voce o sovrascrivere una rotta preesistente senza dover effettuare una cancellazione manuale preventiva.

3. **Risoluzione dell'interfaccia:** converte il nome testuale dell'interfaccia nel rispettivo indice numerico di sistema (`if_idx`) tramite la funzione `if_nametoindex()`.
4. **Conversione della maschera e inoltro:** calcola la lunghezza del prefisso CIDR a partire dalla maschera di rete tramite l'utility `mask_to_prefix()` e delega la formattazione e l'invio del messaggio Netlink a `netlink_send_route_request()`.

4 Test di funzionamento

4.1 Topologia della rete di test



Nodo	Interfaccia	Indirizzo IP	Subnet Associata / Note
RA	eth0	10.12.10.254/24	LAN PC1 (10.12.10.0/24)
	eth4	10.12.1.1/30	Link RA–RB (10.12.1.0/30)
	eth2	10.12.4.1/30	Link RA–RC (10.12.4.0/30)
	eth3	10.12.3.1/30	Link RA–RD (10.12.3.0/30)
RB	eth0	10.12.1.2/30	Link RA–RB (10.12.1.0/30)
	eth1	10.12.2.2/30	Link RB–RC (10.12.2.0/30)
RD	eth0	10.12.3.2/30	Link RA–RD (10.12.3.0/30)
	eth2	10.12.5.2/30	Link RD–RC (10.12.5.0/30)
RC	eth1	10.12.2.1/30	Link RB–RC (10.12.2.0/30)
	eth0	10.12.4.2/30	Link RA–RC (10.12.4.0/30)
	eth3	10.12.5.1/30	Link RD–RC (10.12.5.0/30)
	eth2	10.12.20.254/24	LAN PC2 (10.12.20.0/24)
PC1	eth0	10.12.10.99/24	Gateway: 10.12.10.254
PC2	eth0	10.12.20.99/24	Gateway: 10.12.20.254

4.2 Test di avvio e convergenza iniziale

4.2.1 Avvio dei nodi

La prima fase di test consiste nella verifica del corretto funzionamento dell'applicazione sui singoli nodi della topologia. Come descritto precedentemente, dai log stampati è possibile visualizzare l'esecuzione delle operazioni di inizializzazione del protocollo e delle sue strutture:

```
root@RA:/app# ./rip-in-c
RIPinC Daemon starting...
Loaded network: 10.12.10.0/24
Loaded network: 10.12.4.0/30
Loaded network: 10.12.1.0/30
Loaded network: 10.12.3.0/30
RIP enabled on: eth0 (10.12.10.254)
RIP enabled on: eth2 (10.12.4.1)
RIP enabled on: eth3 (10.12.3.1)
RIP enabled on: eth4 (10.12.1.1)
Socket created. Listening on 224.0.0.9:520
[INIT] Added local interface eth0 to routing table.
[INIT] Added local interface eth2 to routing table.
[INIT] Added local interface eth4 to routing table.
[INIT] Added local interface eth3 to routing table.
```

RIP DATABASE						
Network	Netmask	Next Hop	Metric	Iface	Type	Age(s)
10.12.10.0	255.255.255.0	0.0.0.0	1	eth0	Local	0
10.12.4.0	255.255.255.252	0.0.0.0	1	eth2	Local	0
10.12.1.0	255.255.255.252	0.0.0.0	1	eth4	Local	0
10.12.3.0	255.255.255.252	0.0.0.0	1	eth3	Local	0

Dall'output iniziale del database si evince chiaramente come **RA** sia a conoscenza solo delle reti direttamente collegate. In questa fase non è presente alcuna rotta verso le reti remote, evidenziando la necessità della fase di scambio degli annunci per raggiungere la convergenza.

4.2.2 Convergenza

A seguito dell'avvio simultaneo di tutti i nodi, inizia lo scambio dei messaggi RIP. Il processo sul router RA riceve gli aggiornamenti dai vicini diretti e aggiorna dinamicamente il proprio RIP Database applicando la logica RIP e inviando tempestivamente i Triggered Update.

Di seguito, è riportata la descrizione, delle fasi attraversate da RA:

Apprendimento delle rotte da RD e primo Triggered Update

RA riceve inizialmente un pacchetto di update da RD tramite l'interfaccia eth3. Analizzando il pacchetto, viene individuata una sottorete remota localmente sconosciuta 10.12.5.0/30 e la si registra con metrica pari a 2.

```
[RECV] Received 44 bytes from 10.12.3.2
Received an Update from 10.12.3.2 with 2 routes. Checking routes...
[RIP] Learned NEW route: 10.12.5.0 via 10.12.3.2 (Metric: 2)
```

RIP DATABASE						
Network	Netmask	Next Hop	Metric	Iface	Type	Age(s)
10.12.10.0	255.255.255.0	0.0.0.0	1	eth0	Local	0
10.12.4.0	255.255.255.252	0.0.0.0	1	eth2	Local	0
10.12.1.0	255.255.255.252	0.0.0.0	1	eth4	Local	0
10.12.3.0	255.255.255.252	0.0.0.0	1	eth3	Local	0
10.12.5.0	255.255.255.252	10.12.3.2	2	eth3	RIP	0

```
[TRIGGERED] Sending immediate RIP update after topology change...
[SEND] Periodic Update sent on interface eth0
[SEND] Periodic Update sent on interface eth2
[SEND] Periodic Update sent on interface eth4
[SEND] Periodic Update sent on interface eth3
```

Come evidenziato dal log, la modifica del database causa l'invio di un Triggered Update per accelerare la convergenza ed evitare l'attesa del timer periodico standard.

Apprendimento della LAN remota dal Router RC

Successivamente, RA riceve un altro update proveniente da RC sull'interfaccia eth2. Attraverso questo annuncio, il router apprende l'esistenza della rete locale del client PC2 (10.12.20.0/24) e della subnet 10.12.2.0/30, entrambe registrate con metrica 2 e Next Hop 10.12.4.2.

```
[RECV] Received 104 bytes from 10.12.4.2
Received an Update from 10.12.4.2 with 5 routes. Checking routes...
[RIP] Learned NEW route: 10.12.20.0 via 10.12.4.2 (Metric: 2)
[RIP] Learned NEW route: 10.12.2.0 via 10.12.4.2 (Metric: 2)
```

RIP DATABASE						
Network	Netmask	Next Hop	Metric	Iface	Type	Age(s)
10.12.10.0	255.255.255.0	0.0.0.0	1	eth0	Local	0
10.12.4.0	255.255.255.252	0.0.0.0	1	eth2	Local	0
10.12.1.0	255.255.255.252	0.0.0.0	1	eth4	Local	0
10.12.3.0	255.255.255.252	0.0.0.0	1	eth3	Local	0
10.12.5.0	255.255.255.252	10.12.3.2	2	eth3	RIP	0
10.12.20.0	255.255.255.0	10.12.4.2	2	eth2	RIP	0
10.12.2.0	255.255.255.252	10.12.4.2	2	eth2	RIP	0

Ricezione da RB e stato finale a convergenza avvenuta

Infine, viene ricevuto l'update proveniente da RB su eth4, che porta RA alla convergenza, avendo le rotte per tutte le sottoreti annunciate nella topologia.

```
[RECV] Received 144 bytes from 10.12.1.2
Received an Update from 10.12.1.2 with 7 routes. Checking routes...
```

RIP DATABASE						
Network	Netmask	Next Hop	Metric	Iface	Type	Age(s)
10.12.10.0	255.255.255.0	0.0.0.0	1	eth0	Local	0
10.12.4.0	255.255.255.252	0.0.0.0	1	eth2	Local	0
10.12.1.0	255.255.255.252	0.0.0.0	1	eth4	Local	0
10.12.3.0	255.255.255.252	0.0.0.0	1	eth3	Local	0
10.12.5.0	255.255.255.252	10.12.3.2	2	eth3	RIP	0
10.12.20.0	255.255.255.0	10.12.4.2	2	eth2	RIP	0
10.12.2.0	255.255.255.252	10.12.4.2	2	eth2	RIP	0

A convergenza raggiunta, il database mostra l'efficacia del calcolo dei cammini minimi. Infatti, la destinazione 10.12.20.0/24 (LAN del client PC2) viene instradata direttamente tramite il link verso RC, scartando i percorsi alternativi a metrica superiore (3 hop via RD o RB).

4.3 Gestione dei guasti

4.3.1 Caduta del link RA–RC

Per dimostrare il comportamento di RIP a seguito dell'interruzione di un link, è stata valutata la sua efficacia simulando la caduta del collegamento centrale tra RA e RC. Allo stato attuale della convergenza, effettuando un traceroute da PC1 a PC2 è possibile visualizzare il percorso di routing utilizzato per la comunicazione:

```
root@PC1:~# traceroute 10.12.20.99 -n
traceroute to 10.12.20.99 (10.12.20.99), 30 hops max, 60 byte packets
 1  10.12.10.254  0.481 ms  0.458 ms  0.446 ms
 2  10.12.4.2    1.214 ms  1.169 ms  1.157 ms
 3  10.12.20.99  2.044 ms  1.994 ms  1.993 ms
```

Il guasto è stato prodotto spegnendo l'interfaccia eth0 su RC tramite il comando:

```
root@RC:~# ip link set eth0 down
```

Fase 1: Reazione locale immediata su RC

Non appena l'interfaccia fisica passa nello stato di DOWN, l'applicazione intercetta la disattivazione del link tramite Netlink e applica il meccanismo di *Route Poisoning*. Le rotte dipendenti da eth0 vengono immediatamente contrassegnate come irraggiungibili e, come illustrato nel listato sottostante, una volta ricevuto l'annuncio da RB, le rotte avvelenate vengono reimpostate come raggiungibili e la rotta relativa alla rete di cui faceva parte l'interfaccia spenta cambia tipologia e diventa una rotta dinamica RIP:

```
[LINK] Interface eth0 is down, poisoning dependent route 10.12.10.0
[LINK] Interface eth0 is down, poisoning dependent route 10.12.1.0
[LINK] Interface eth0 is down, poisoning local route 10.12.4.0/30
[RECV] Received 184 bytes from 10.12.2.2
Received an Update from 10.12.2.2 with 9 routes. Checking routes...
[RIP] Route IMPROVED for 10.12.10.0 via 10.12.2.2 (Metric: 16 -> 3)
[RIP] Route IMPROVED for 10.12.1.0 via 10.12.2.2 (Metric: 16 -> 3)
[RIP] Route IMPROVED for 10.12.4.0 via 10.12.5.2 (Metric: 16 -> 3)
```

RIP DATABASE						
Network	Netmask	Next Hop	Metric	Iface	Type	Age(s)
10.12.20.0	255.255.255.0	0.0.0.0	1	eth2	Local	0
10.12.4.0	255.255.255.252	10.12.5.2	3	eth3	RIP	0
10.12.2.0	255.255.255.252	0.0.0.0	1	eth1	Local	0
10.12.5.0	255.255.255.252	0.0.0.0	1	eth3	Local	0
10.12.3.0	255.255.255.252	10.12.5.2	2	eth3	RIP	0
10.12.10.0	255.255.255.0	10.12.2.2	3	eth1	RIP	0
10.12.1.0	255.255.255.252	10.12.2.2	3	eth1	RIP	0

Fase 2: Aging delle rotte sul Router RA

Il router RA non è ancora a conoscenza dello spegnimento dell'interfaccia di RC, di conseguenza mantiene attive le rotte che hanno come next hop RC. Quello di cui è a conoscenza è che gli update provenienti da RC iniziano a mancare, quindi il campo Age delle rotte dipendenti da RC inizia ad aumentare:

RIP DATABASE						
Network	Netmask	Next Hop	Metric	Iface	Type	Age(s)
10.12.10.0	255.255.255.0	0.0.0.0	1	eth0	Local	0
10.12.4.0	255.255.255.252	0.0.0.0	1	eth2	Local	0
10.12.1.0	255.255.255.252	0.0.0.0	1	eth4	Local	0
10.12.3.0	255.255.255.252	0.0.0.0	1	eth3	Local	0
10.12.20.0	255.255.255.0	10.12.4.2	2	eth2	RIP	58
10.12.2.0	255.255.255.252	10.12.4.2	2	eth2	RIP	58
10.12.5.0	255.255.255.252	10.12.3.2	2	eth3	RIP	1

Fase 3: Scadenza del Timer e avvelenamento della rotta

Raggiunta la soglia di timeout senza ricevere conferme di attività da parte di RC, il timer interno del processo dichiara le rotte scadute, impostandone forzatamente la metrica a 16:

```
[TIMER] Route expired for 10.12.20.0/24, poisoning metric to 16
[TIMER] Route expired for 10.12.2.0/30, poisoning metric to 16
```

RIP DATABASE						
Network	Netmask	Next Hop	Metric	Iface	Type	Age(s)
10.12.10.0	255.255.255.0	0.0.0.0	1	eth0	Local	0
10.12.4.0	255.255.255.252	0.0.0.0	1	eth2	Local	0
10.12.1.0	255.255.255.252	0.0.0.0	1	eth4	Local	0
10.12.3.0	255.255.255.252	0.0.0.0	1	eth3	Local	0
10.12.20.0	255.255.255.0	10.12.4.2	16	eth2	RIP	60
10.12.2.0	255.255.255.252	10.12.4.2	16	eth2	RIP	60
10.12.5.0	255.255.255.252	10.12.3.2	2	eth3	RIP	3

Fase 4: Riconvergenza su cammino alternativo (via RD)

A questo punto, non appena giunge l'annuncio periodico di RD, l'algoritmo rileva che le reti appena avvelenate sono ancora raggiungibili attraverso il ramo superiore con metrica 3. Poichè la metrica proposta da RD (3) è inferiore al 16 attualmente impostato, il router registra l'evento [RIP] Route IMPROVED, riassegnando l'inoltro all'interfaccia eth3 con *Next Hop* 10.12.3.2.

```
[RECV] Received an Update from 10.12.3.2 with 9 routes. Checking routes...
[RIP] Route IMPROVED for 10.12.20.0 via 10.12.3.2 (Metric: 16 -> 3)
[RIP] Route IMPROVED for 10.12.2.0 via 10.12.3.2 (Metric: 16 -> 3)
```

RIP DATABASE						
Network	Netmask	Next Hop	Metric	Iface	Type	Age(s)
10.12.10.0	255.255.255.0	0.0.0.0	1	eth0	Local	0
10.12.4.0	255.255.255.252	0.0.0.0	1	eth2	Local	0
10.12.1.0	255.255.255.252	0.0.0.0	1	eth4	Local	0
10.12.3.0	255.255.255.252	0.0.0.0	1	eth3	Local	0
10.12.20.0	255.255.255.0	10.12.3.2	3	eth3	RIP	0
10.12.2.0	255.255.255.252	10.12.3.2	3	eth3	RIP	0
10.12.5.0	255.255.255.252	10.12.3.2	2	eth3	RIP	0

A seguito dell'aggiornamento, il traffico generato da **PC1** verso **PC2** riprende a scorrere senza perdite definitive, attraversando il percorso alternativo RA -> RD -> RC -> PC2:

```
root@PC1:~# traceroute 10.12.20.99 -n
traceroute to 10.12.20.99 (10.12.20.99), 30 hops max, 60 byte packets
 1  10.12.10.254  0.448 ms  0.354 ms  0.339 ms
 2  10.12.3.2    1.477 ms  1.435 ms  1.416 ms
 3  10.12.5.1    2.177 ms  2.095 ms  2.047 ms
 4  10.12.20.99  2.189 ms  2.208 ms  2.188 ms
```

4.3.2 Ripristino del link RA-RC

A questo punto, è il momento di simulare il ripristino del canale di comunicazione appena reso inutilizzabile riattivando l'interfaccia su RC:

```
root@RC:~# ip link set eth0 up
```

Non appena l'interfaccia torna operativa, RC riprende la trasmissione periodica dei datagrammi RIP e, come illustrato di seguito, RA riceve i suoi update:

```
[RECV] Received 184 bytes from 10.12.4.2
Received an Update from 10.12.4.2 with 9 routes. Checking routes...
[RIP] Route IMPROVED for 10.12.20.0 via 10.12.4.2 (Metric: 3 -> 2)
[RIP] Route IMPROVED for 10.12.2.0 via 10.12.4.2 (Metric: 3 -> 2)
```

RIP DATABASE						
Network	Netmask	Next Hop	Metric	Iface	Type	Age(s)
10.12.10.0	255.255.255.0	0.0.0.0	1	eth0	Local	0
10.12.4.0	255.255.255.252	0.0.0.0	1	eth2	Local	0
10.12.1.0	255.255.255.252	0.0.0.0	1	eth4	Local	0
10.12.3.0	255.255.255.252	0.0.0.0	1	eth3	Local	0
10.12.20.0	255.255.255.0	10.12.4.2	2	eth2	RIP	0
10.12.2.0	255.255.255.252	10.12.4.2	2	eth3	RIP	0
10.12.5.0	255.255.255.252	10.12.3.2	2	eth3	RIP	4

Questo test dimostra la capacità della logica operativa di ripristinare sempre il percorso migliore non appena sia disponibile.

4.3.3 Caduta della rete 10.12.20.0/24

L'ultimo scenario di cui viene testato il comportamento, è quello in cui una rete diventi totalmente irraggiungibile per l'assenza di percorsi alternativi. A tale scopo, è stata disattivata l'interfaccia `eth2` di RC, corrispondente alla LAN del client PC2:

```
root@RC:~# ip link set eth2 down
```

Come evidenziato di seguito, RC rileva immediatamente la caduta del link tramite Netlink e avvelena la rotta impostando la sua metrica a 16:

```
[LINK] Interface eth2 is down, poisoning local route 10.12.20.0/24
```

RIP DATABASE						
Network	Netmask	Next Hop	Metric	Iface	Type	Age(s)
10.12.20.0	255.255.255.0	0.0.0.0	16	eth2	Local	0
10.12.2.0	255.255.255.252	0.0.0.0	1	eth1	Local	0
10.12.5.0	255.255.255.252	0.0.0.0	1	eth3	Local	0
10.12.4.0	255.255.255.252	0.0.0.0	1	eth0	Local	0
10.12.3.0	255.255.255.252	10.12.5.2	2	eth3	RIP	2
10.12.10.0	255.255.255.0	10.12.4.1	2	eth0	RIP	4

L'avvelenamento viene immediatamente propagato tramite gli update verso i vicini. Prendendo in esempio RA, la rotta viene aggiornata impostando la metrica da 2 a 16 e viene rimossa dalla tabella del kernel per impedire l'inoltro di traffico a vuoto:

```
[RECV] Received 184 bytes from 10.12.4.2
Received an Update from 10.12.4.2 with 9 routes. Checking routes...
[RIP] Route CHANGED for 10.12.20.0: old metric 2, new metric 16
```

RIP DATABASE						
Network	Netmask	Next Hop	Metric	Iface	Type	Age(s)
10.12.10.0	255.255.255.0	0.0.0.0	1	eth0	Local	0
10.12.4.0	255.255.255.252	0.0.0.0	1	eth2	Local	0
10.12.1.0	255.255.255.252	0.0.0.0	1	eth4	Local	0
10.12.3.0	255.255.255.252	0.0.0.0	1	eth3	Local	0
10.12.20.0	255.255.255.0	10.12.4.2	16	eth2	RIP	0
10.12.2.0	255.255.255.252	10.12.4.2	2	eth2	RIP	3
10.12.5.0	255.255.255.252	10.12.3.2	2	eth3	RIP	1

A questo punto, una volta che la metrica viene impostata a 16, ne viene memorizzato il timestamp dando inizio alla fase Garbage Collection. Come prescritto dallo standard, la permanenza nel database della rotta con metrica infinita serve ad assicurare che l'avvelenamento raggiunga l'intero dominio di routing evitando disallineamenti nella topologia. Al termine del timer, il garbage collector elimina definitivamente la rotta dal database:

```
[GC] Removing expired route 10.12.20.0/24 from database
```

RIP DATABASE						
Network	Netmask	Next Hop	Metric	Iface	Type	Age(s)
10.12.10.0	255.255.255.0	0.0.0.0	1	eth0	Local	0
10.12.4.0	255.255.255.252	0.0.0.0	1	eth2	Local	0
10.12.1.0	255.255.255.252	0.0.0.0	1	eth4	Local	0
10.12.3.0	255.255.255.252	0.0.0.0	1	eth3	Local	0
10.12.5.0	255.255.255.252	10.12.3.2	2	eth3	RIP	6
10.12.2.0	255.255.255.252	10.12.4.2	2	eth2	RIP	0

4.3.4 Cattura del traffico

Di seguito viene proposta la cattura tramite Wireshark di un Update inviato da RA a RC.

```
▶ Frame 12: 186 bytes on wire (1488 bits), 186 bytes captured (1488 bits) on interface eth0, id 0
▶ Ethernet II, Src: Yamashita 00:01:02 (00:50:01:00:01:02), Dst: IPv4mcast_09 (01:00:5e:00:00:09)
▶ Internet Protocol Version 4, Src: 10.12.4.1, Dst: 224.0.0.9
▶ User Datagram Protocol, Src Port: 520, Dst Port: 520
▼ Routing Information Protocol
  Command: Response (2)
  Version: RIPv2 (2)
  ▶ IP Address: 10.12.10.0, Metric: 1
  ▶ IP Address: 10.12.4.0, Metric: 1
  ▶ IP Address: 10.12.1.0, Metric: 1
  ▶ IP Address: 10.12.3.0, Metric: 1
  ▶ IP Address: 10.12.5.0, Metric: 2
  ▶ IP Address: 10.12.20.0, Metric: 16
  ▼ IP Address: 10.12.2.0, Metric: 16
    Address Family: IP (2)
    Route Tag: 0
    IP Address: 10.12.2.0
    Netmask: 255.255.255.252
    Next Hop: 0.0.0.0
    Metric: 16
```

La cattura dimostra che Wireshark riconosce il formato del pacchetto RIP e ne propone la comoda visualizzazione ad albero di tutti i campi di cui è composto. Inoltre, si può anche notare il funzionamento di Split-Horizon, dal momento che le metriche delle due rotte apprese da RC sono impostate a 16.

5 Organizzazione e struttura del codice sorgente

La struttura del codice è di seguito rappresentata:

```
project-root/
├── src/
│   ├── main.c
│   ├── network.c
│   ├── network.h
│   ├── rip-protocol-specs.h
│   ├── routing.c
│   └── routing.h
├── docker/
│   ├── configs/
│   │   └── router-X.conf
│   ├── Dockerfile
│   └── docker-compose.yaml
└── Makefile
```

Per ogni elemento viene fornita relativa descrizione:

- `src`: è la cartella che contiene tutti i file sorgente necessari alla compilazione dell'applicazione;
- `docker`: contiene il `docker-compose.yaml` e il `Dockerfile` necessari per l'avvio dei nodi tramite Docker. Al suo interno è anche presente una cartella che è destinata a contenere le configurazioni dei vari router che vengono avviati tramite Docker.
- `Makefile`: è il file di configurazione utilizzato dall'utility `make`.

6 Installazione e avvio

6.1 Installazione manuale (Makefile)

Il software può essere compilato ed eseguito direttamente sull'ambiente host (sistema operativo basato su Linux).

6.1.1 Requisiti di sistema

Per procedere al download del codice sorgente e la compilazione del binario, è necessario disporre dei seguenti pacchetti:

- **Build tool:** git, gcc e make.

Su distribuzioni basate su Debian/Ubuntu, tali componenti possono essere installati tramite il gestore di pacchetti apt:

```
sudo apt-get update
sudo apt-get install -y git gcc make
```

6.1.2 Clonazione della repository e compilazione

Una volta verificata la sussistenza dei requisiti, è possibile compilare il progetto eseguendo il seguente comando:

```
make
```

Al termine del processo di compilazione, nella directory corrente verrà generato il file binario eseguibile `rip-in-c`.

6.1.3 Configurazione del nodo

Affinchè l'applicazione funzioni correttamente, è necessario che nella stessa cartella dell'eseguibile sia presente il file di configurazione `router.conf`, contenente l'elenco delle reti su cui abilitare il protocollo. Nella repository è fornito un esempio denominato `router.conf.example`.

6.1.4 Esecuzione e pulizia

Per avviare il processo RIP è sufficiente eseguire il binario generato con privilegi di root, dal momento che l'applicazione necessita di interagire con strutture a basso livello del sistema e lavorare con i socket:

```
./rip-in-c
```

Per rimuovere i file oggetto intermedi e l'eseguibile compilato, è possibile utilizzare:

```
make clean
```

6.2 Docker

Per permettere a chi non ha a disposizione una VM per ogni nodo e semplificare lo sviluppo e configurazione dell'intera topologia, nella repository è proposta la configurazione per l'avvio in ambiente containerizzato basato su Docker e Docker Compose.

6.2.1 Immagine Docker

Il `Dockerfile` adotta un approccio *multi-stage build* per separare la fase di compilazione dall'ambiente di runtime:

Stage di build

Utilizza un'immagine di base `ubuntu:22.04` con `gcc` e `make` per compilare il codice sorgente C.

Stage di runtime

Copia esclusivamente il binario compilato `rip-in-c`, installando solo le utility necessarie al funzionamento e al debugging di rete (`iproute`, `iputils-ping`, `tcpdump`, `net-tools`).

6.2.2 Configurazione dell'ambiente con Docker Compose

Il file `docker-compose.yml` descrive l'intera rete riproducendo una topologia con tre nodi:

Isolamento delle subnet

Definisce reti Docker dedicate assegnando subnet e indirizzi IPv4 statici a ciascuna interfaccia dei container.

Abilitazione dell'IP Forwarding

Tramite la direttiva `sysctl`: `[net.ipv4.ip_forward=1]`, i container vengono configurati a livello kernel per permettere l'inoltro del traffico in ingresso usando la propria tabella di routing.

Privilegi di rete

La capability Linux `NET_ADMIN` è assegnata a router e client per consentire la manipolazione delle tabelle di routing del kernel.

Configurazione dei nodi

Tramite `bind mount` di volume, ciascun nodo riceve il proprio file di configurazione con le reti su cui abilitare il processo RIP.

6.2.3 Avvio ed esecuzione

L'intera infrastruttura (router e host client) può essere compilata e avviata con un unico comando dalla cartella `docker`:

```
docker compose up --build -d
```

Per arrestare la topologia e ripulire le reti allocate:

```
docker compose down
```

7 Sviluppi futuri

Nonostante l'attuale implementazione di RIP fornisca un sistema di routing funzionante e reattivo ai cambi di topologia, il sistema presenta ampi margini di estensione per avvicinarsi sempre di più alle specifiche dello standard definito in RFC. Tra i principali punti da sviluppare si individuano:

Supporto completo a RIPv2 (RFC 2453)

L'attuale versione gestisce un formato base del pacchetto, ma l'integrazione completa dei nuovi campi introdotti nella versione 2 permetterebbe di sfruttare a pieno le potenzialità del protocollo:

- Route Tag per distinguere le rotte interne dell'AS da quelle distribuite da protocolli esterni.
- Next-Hop esplicito per ottimizzare l'instradamento quando due router condividono la stessa rete broadcast con un terzo nodo.

Sicurezza e Autenticazione dei pacchetti

Lo standard permette l'utilizzo di autenticazione mediante MD5 o pre-shared key, in modo da prevenire l'iniezione malevole di rotte fasulle (Route Injection / Spoofing).

Interfaccia di Management

Lo sviluppo di un'interfaccia a riga di comando che permetta all'amministratore di rete di interagire con il processo porterebbe l'attuale implementazione ad uno step superiore.

Estensione a IPv6 (RIPng - RFC 2080)

Per supportare IPv6 (già ampiamente usato) bisognerebbe portare avanti uno sviluppo di non poco conto. Bisognerebbe adattare le strutture dati interne, il socket layer e il parser dei pacchetti per operare su indirizzi IPv6 a 128 bit.